目 录

一、实验过程
- 1. RDT 1.0
- 2. RDT 2.x
- 3. RDT 3.0
- 4. Select Response
- 5. TCP
- 6. TCP Tahoe
- 7. TCP Reno

二、实验总结
- 1. 未完成的项目
- 2. 迭代开发
- 3. 已解决的主要问题
- 4. 系统性建议

一、实验过程

本实验基于老师提供的RDT1.0代码，在 **macOS** 操作系统环境下完成，使用 **Visual Studio Code** 作为代码编辑与调试工具，采用 **Java（OpenJDK 17）** 作为开发与运行环境。项目开发过程中引入 **Git** 进行版本控制，以实现代码的迭代开发与管理，保证实验过程的规范性与可追溯性。

1. RDT 1.0

RDT1.0 假设底层信道完全可靠：不丢包、不出错、不延迟。因此协议侧不需要引入校验、超时或复杂确认策略，只需完成“发送—接收—确认”流程即可。这里配置好环境以后直接运行提供好的代码，观察其运行结果，log文件如下：

```
CLIENT HOST TOTAL   SUC_RATIO   NORMAL  WRONG   LOSS    DELAY
10.140.222.114:19001    1000    100.00% 1000    0   0   0
    2025-12-10 10:25:50:900 CST DATA_seq: 1     ACKed
    2025-12-10 10:25:50:924 CST DATA_seq: 101       ACKed
    2025-12-10 10:25:50:941 CST DATA_seq: 201       ACKed
    2025-12-10 10:25:50:957 CST DATA_seq: 301       ACKed
    2025-12-10 10:25:50:972 CST DATA_seq: 401       ACKed
    2025-12-10 10:25:50:988 CST DATA_seq: 501       ACKed
    2025-12-10 10:25:51:003 CST DATA_seq: 601       ACKed
    2025-12-10 10:25:51:017 CST DATA_seq: 701       ACKed
    2025-12-10 10:25:51:030 CST DATA_seq: 801       ACKed
    2025-12-10 10:25:51:044 CST DATA_seq: 901       ACKed
    2025-12-10 10:25:51:059 CST DATA_seq: 1001      ACKed
    2025-12-10 10:25:51:073 CST DATA_seq: 1101      ACKed
    2025-12-10 10:25:51:089 CST DATA_seq: 1201      ACKed
    2025-12-10 10:25:51:103 CST DATA_seq: 1301      ACKed
    2025-12-10 10:25:51:117 CST DATA_seq: 1401      ACKed
    2025-12-10 10:25:51:130 CST DATA_seq: 1501      ACKed
    2025-12-10 10:25:51:143 CST DATA_seq: 1601      ACKed
```

图 1 发送端

```
10.140.222.114:19002    1000    100.00% 1000    0   0   0
    2025-12-10 10:25:50:909 CST ACK_ack: 1
    2025-12-10 10:25:50:926 CST ACK_ack: 101
    2025-12-10 10:25:50:942 CST ACK_ack: 201
    2025-12-10 10:25:50:958 CST ACK_ack: 301
    2025-12-10 10:25:50:974 CST ACK_ack: 401
    2025-12-10 10:25:50:989 CST ACK_ack: 501
    2025-12-10 10:25:51:004 CST ACK_ack: 601
    2025-12-10 10:25:51:018 CST ACK_ack: 701
    2025-12-10 10:25:51:031 CST ACK_ack: 801
    2025-12-10 10:25:51:045 CST ACK_ack: 901
    2025-12-10 10:25:51:060 CST ACK_ack: 1001
    2025-12-10 10:25:51:074 CST ACK_ack: 1101
    2025-12-10 10:25:51:089 CST ACK_ack: 1201
    2025-12-10 10:25:51:103 CST ACK_ack: 1301
    2025-12-10 10:25:51:117 CST ACK_ack: 1401
    2025-12-10 10:25:51:130 CST ACK_ack: 1501
    2025-12-10 10:25:51:144 CST ACK_ack: 1601
    2025-12-10 10:25:51:158 CST ACK_ack: 1701
    2025-12-10 10:25:51:170 CST ACK_ack: 1801
```

图 2 接收端

每个数据段发送后都能及时收到确认，recvData 连续完整。

2. RDT 2.x

RDT 2.0：引入校验和以解决“位错（bit error）”

RDT 2.x 在 RDT 1.0 的基础上放宽了信道假设，允许数据包在传输过程中发生位错，但不考虑丢包与延迟问题。为此，协议需要具备检测数据包错误并触发重传的能力。

```java
public static short computeChkSum(TCP_PACKET tcpPack) {
  TCP_HEADER tcpHeader = tcpPack.getTcpH();
  int seq = tcpHeader.getTh_seq();
  int ack = tcpHeader.getTh_ack();
  TCP_SEGMENT tcp_SEGMENT = tcpPack.getTcpS();
  int[] data = tcp_SEGMENT.getData();

  CRC32 crc32 = new CRC32();
  crc32.update(seq);
  crc32.update(ack);
  for(int i:data){
    crc32.update(i);
```

```
    }
    int checkSum = (int)crc32.getValue();

    // int checkSum = 0;
    return (short) checkSum;
  }
```

在 RDT 2.0 阶段，系统实现了**基于 CRC32 的校验和计算方法**。发送端在构造 TCP 报文段时，对 TCP 首部中的序号、确认号以及数据字段进行 CRC32 计算，并将计算结果写入首部的校验和字段。

接收端在收到数据包后，重新计算校验和并与报文段中携带的校验和字段进行比较，以判断数据包在传输过程中是否发生位错。

该校验机制能够有效检测数据内容或关键首部字段的单比特或多比特错误。

RDT 2.2：差错检测

在 RDT 2.2 中，协议采用仅使用 ACK 的方式完成差错控制，不再使用 NAK 报文。当接收端检测到数据包校验和错误时，不直接返回否定确认，而是返回对“上一个正确接收数据包”的确认（重复 ACK）。

接收端同时维护一个期望接收的数据包序号，仅当接收到的数据包序号与期望序号一致时，才将数据交付给上层应用；对于重复或错误的数据包，不进行重复交付。

发送端在停等状态下等待确认报文段，当收到的 ACK 与当前发送的数据包序号不匹配时，认为该数据包未被正确接收，并重传当前数据包。

```
public class TCP_Receiver extends TCP_Receiver_ADT {

  private TCP_PACKET ackPack; //回复的ACK报文段
  // RDT2.1/2.2，序列号sequence0/1交替
  int sequence=0;//用于记录当前待接收的包序号，注意包序号不完全是

  /*构造函数*/
  public TCP_Receiver() {
    super();  //调用超类构造函数
    super.initTCP_Receiver(this); //初始化TCP接收端
  }

  @Override
  //接收到数据报：检查校验和，设置回复的ACK报文段
  public void rdt_recv(TCP_PACKET recvPack) {
    //检查校验码，生成ACK
    int dataLength = recvPack.getTcpS().getData().length;
    int dataSequence = (recvPack.getTcpH().getTh_seq() - 1) / dataLength;
    if(CheckSum.computeChkSum(recvPack) == recvPack.getTcpH().getTh_sum()) {
      //生成ACK报文段（设置确认号）
      tcpH.setTh_ack(recvPack.getTcpH().getTh_seq());
      ackPack = new TCP_PACKET(tcpH, tcpS, recvPack.getSourceAddr());
      tcpH.setTh_sum(CheckSum.computeChkSum(ackPack));
      //回复ACK报文段
      reply(ackPack);

      //将接收到的正确有序的数据插入data队列，准备交付
```

```
        if (dataSequence == sequence){
          dataQueue.add(recvPack.getTcpS().getData());
          sequence++;
        }
      }else{
        System.out.println("Recieve Computed: "+CheckSum.computeChkSum(recvPack));
        System.out.println("Recieved Packet"+recvPack.getTcpH().getTh_sum());
        System.out.println("Problem: Packet Number: "+recvPack.getTcpH().getTh_seq()+" +
InnerSeq:  "+sequence);
        tcpH.setTh_ack((sequence-1) * dataLength+1); // 不再使用NAK
        ackPack = new TCP_PACKET(tcpH, tcpS, recvPack.getSourceAddr());
        tcpH.setTh_sum(CheckSum.computeChkSum(ackPack));
        //回复ACK报文段
        reply(ackPack);
      }

      System.out.println();


      //交付数据（每20组数据交付一次）
      if(dataQueue.size() == 20)
        deliver_data();
    }
```

现在我们的th_seq是字节流序号，我们要得到对应的分组编号即dataIndex，对应的seq的相关关系为

$$seq = dataIndex * dataLength + 1 \tag{1}$$

所以重传上一个正确接收的数据包dataIndex可以通过公式计算来得到。

接收端对接收到的数据包计算出对应的dataIndex，然后检查校验和，如果校验和正确，则包没有出现位错，生成对应的ACK报文段，设置确认号为接收到的数据包的序列号，然后回复ACK报文段；若校验和错误，则生成ACK报文段，设置确认号为上一个正确接收到的数据包的序列号，然后回复ACK报文段。发送端看到重复ACK，重传当前分组。

将发送方和接收放设置为`tcpH.setTh_eflag((byte) 1)`，允许信道出错，运行RDT2.0代码结果如下：

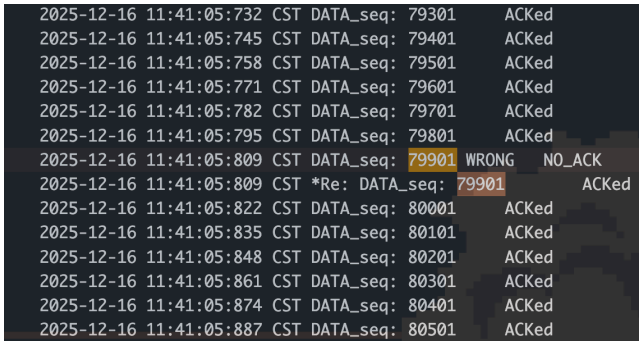


图 3 发送端

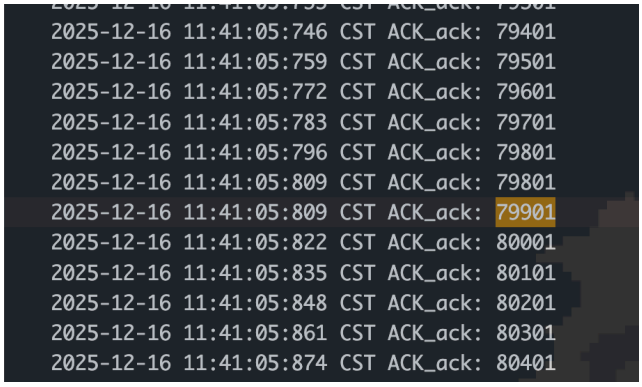


图 4 接收端

在log中可以看到发送了一个错误包79901后没有收到对应的ACK，重复收到了79801，然后重传79901。最后的recvDate中还是100000个，说明没有出现丢失或重复，逻辑正确。

3. RDT 3.0

RDT 3.0 进一步放宽信道假设，允许数据包或确认包在传输过程中发生丢失或延迟。在仅依赖校验和与重复 ACK 的情况下，发送端可能因收不到确认而无法判断数据包状态，因此需要引入超时重传机制。

发送端修改如下

```
// 加入计时器
private UDT_Timer timer;

// 新增方法启动定时器
private void startTimer(){
  timer = new UDT_Timer();
  timer.schedule(
    new UDT_RetransTask(client, tcpPack),
    // 3s后开始重传，每3s重传一次
    3000,
    3000
  );
}

// 新增方法停用定时器
private void stopTimer(){
  if (timer != null){
    timer.cancel();
    timer = null;
  }
}

@Override
//可靠发送（应用层调用）：封装应用层数据，产生TCP数据报；需要修改
public void rdt_send(int dataIndex, int[] appData) {


  //生成TCP数据报（设置序号和数据字段/校验和),注意打包的顺序
  // 1.设置序号
  tcpH.setTh_seq(dataIndex * appData.length + 1);//包序号设置为字节流号:
  // 2.封装数据
  tcpS.setData(appData);
  tcpPack = new TCP_PACKET(tcpH, tcpS, destinAddr);
  // 3.计算校验和
  tcpH.setTh_sum(CheckSum.computeChkSum(tcpPack));
  tcpPack.setTcpH(tcpH);
  // 4.发送数据包
  udt_send(tcpPack);
  flag = 0;

  // 5.启动定时器
  startTimer();    //新增
  // 6.进入WAIT_ACK状态
  //等待ACK报文
```

```
    //waitACK();
    while (flag==0);
  }
.....
  public void waitACK() {

    // 循环检查ackQueue
    // 循环检查确认号对列中是否有新收到的ACK
    if(!ackQueue.isEmpty()){
      int currentAck=ackQueue.poll();
      // System.out.println("CurrentAck: "+currentAck);
      if (currentAck == tcpPack.getTcpH().getTh_seq()){
        System.out.println("Clear: "+tcpPack.getTcpH().getTh_seq());
        stopTimer(); // 新增
        flag = 1;
        //break;
      }
    }
  }
```

发送端为当前发送的数据包设置唯一的定时器。当发送端发送数据包后启动定时器，若在预设时间内未收到匹配的确认报文段，则由定时器触发重传任务，对当前数据包进行重传。

当发送端收到正确的确认报文段后，立即取消定时器并进入下一轮数据发送，从而避免不必要的重复重传。

该机制保证了在数据包或 ACK 丢失的情况下，发送端仍能通过超时检测恢复传输过程。

在接收端中，仅对校验和正确的数据包进行处理，并维持期望接收的数据包序号：

- 对于序号小于或等于期望序号的数据包（包括重传包），接收端返回确认；
- 仅当接收到的数据包序号等于期望序号时，才将数据交付上层并更新期望序号；
- 对于超前到达的乱序数据包以及校验失败的数据包，接收端直接丢弃且不返回确认。

```
  @Override
  //接收到数据报：检查校验和，设置回复的ACK报文段
  public void rdt_recv(TCP_PACKET recvPack) {
    //检查校验码，生成ACK
    int dataLength = recvPack.getTcpS().getData().length;
    int dataSequence = (recvPack.getTcpH().getTh_seq() - 1) / dataLength;

    // 校验和正确
    if(CheckSum.computeChkSum(recvPack) == recvPack.getTcpH().getTh_sum()) {
      // 对期望序号之前的包都回复ACK
      if(dataSequence <= sequence){
        //生成ACK报文段（设置确认号）
        tcpH.setTh_ack(recvPack.getTcpH().getTh_seq());
        ackPack = new TCP_PACKET(tcpH, tcpS, recvPack.getSourceAddr());
        tcpH.setTh_sum(CheckSum.computeChkSum(ackPack));
        //回复ACK报文段
        reply(ackPack);
```

```
        // 只交付正好是期望序号的数据
        if (dataSequence == sequence){
          dataQueue.add(recvPack.getTcpS().getData());
          sequence = dataSequence+1;
        }
      }
    }

    System.out.println();


    //交付数据（每20组数据交付一次）
    if(dataQueue.size() == 20)
      deliver_data();
  }
```

这里我们直接设置为`tcpH.setTh_eflag((byte) 7)`，允许信道出错、丢包、延迟，运行代码，查看对应的log文件：

```
2025-12-30 10:57:19:184 CST DATA_seq: 21201     ACKed
2025-12-30 10:57:19:198 CST DATA_seq: 21301     ACKed
2025-12-30 10:57:19:213 CST DATA_seq: 21401     ACKed
2025-12-30 10:57:19:227 CST DATA_seq: 21501     ACKed
2025-12-30 10:57:19:241 CST DATA_seq: 21601     ACKed
2025-12-30 10:57:19:256 CST DATA_seq: 21701     ACKed
2025-12-30 10:57:19:270 CST DATA_seq: 21801     ACKed
2025-12-30 10:57:19:283 CST DATA_seq: 21901     ACKed
2025-12-30 10:57:19:297 CST DATA_seq: 22001 WRONG   NO_ACK
2025-12-30 10:57:22:302 CST *Re: DATA_seq: 22001        ACKed
2025-12-30 10:57:22:314 CST DATA_seq: 22101     ACKed
2025-12-30 10:57:22:327 CST DATA_seq: 22201     ACKed
2025-12-30 10:57:22:340 CST DATA_seq: 22301     ACKed
2025-12-30 10:57:22:353 CST DATA_seq: 22401     ACKed
2025-12-30 10:57:22:366 CST DATA_seq: 22501     ACKed
2025-12-30 10:57:22:379 CST DATA_seq: 22601     ACKed
2025-12-30 10:57:22:392 CST DATA_seq: 22701     ACKed
```

图 7 发送端

```
2025-12-30 10:57:19:213 CST ACK_ack: 21401
2025-12-30 10:57:19:228 CST ACK_ack: 21501
2025-12-30 10:57:19:242 CST ACK_ack: 21601
2025-12-30 10:57:19:256 CST ACK_ack: 21701
2025-12-30 10:57:19:270 CST ACK_ack: 21801
2025-12-30 10:57:19:283 CST ACK_ack: 21901
2025-12-30 10:57:22:302 CST ACK_ack: 22001
2025-12-30 10:57:22:314 CST ACK_ack: 22101
2025-12-30 10:57:22:327 CST ACK_ack: 22201
2025-12-30 10:57:22:340 CST ACK_ack: 22301
2025-12-30 10:57:22:353 CST ACK_ack: 22401
2025-12-30 10:57:22:366 CST ACK_ack: 22501
2025-12-30 10:57:22:379 CST ACK_ack: 22601
2025-12-30 10:57:22:392 CST ACK_ack: 22701
```

图 8 接收端

在log中发现对应的22001包出错，没有收到ACK，显示NO_ACK，然后重传了数据包，log中的相差的时间为设定好的3s。

还有接收端发送了延迟（DELAY）的确认情况：

```
2025-12-30 10:58:19:818 CST ACK_ack: 92801
2025-12-30 10:58:19:831 CST ACK_ack: 92901
2025-12-30 10:58:19:844 CST ACK_ack: 93001
2025-12-30 10:58:19:857 CST ACK_ack: 93101
2025-12-30 10:58:19:869 CST ACK_ack: 93201
2025-12-30 10:58:19:882 CST ACK_ack: 93301   DELAY
2025-12-30 10:58:22:887 CST *Re: ACK_ack: 93301
2025-12-30 10:58:22:900 CST ACK_ack: 93401
2025-12-30 10:58:22:913 CST ACK_ack: 93501
2025-12-30 10:58:22:926 CST ACK_ack: 93601
2025-12-30 10:58:22:939 CST ACK_ack: -951415266 WR
2025-12-30 10:58:25:943 CST ACK_ack: 93701
2025-12-30 10:58:25:956 CST ACK_ack: 93801
```

图 9 接收端

```
2025-12-30 10:58:19:793 CST DATA_seq: 92601     ACKed
2025-12-30 10:58:19:804 CST DATA_seq: 92701     ACKed
2025-12-30 10:58:19:817 CST DATA_seq: 92801     ACKed
2025-12-30 10:58:19:830 CST DATA_seq: 92901     ACKed
2025-12-30 10:58:19:843 CST DATA_seq: 93001     ACKed
2025-12-30 10:58:19:856 CST DATA_seq: 93101     ACKed
2025-12-30 10:58:19:868 CST DATA_seq: 93201     ACKed
2025-12-30 10:58:19:881 CST DATA_seq: 93301     NO_ACK      You
2025-12-30 10:58:22:887 CST *Re: DATA_seq: 93301        ACKed
2025-12-30 10:58:22:900 CST DATA_seq: 93401     ACKed
2025-12-30 10:58:22:913 CST DATA_seq: 93501     ACKed
2025-12-30 10:58:22:926 CST DATA_seq: 93601     ACKed
2025-12-30 10:58:22:939 CST DATA_seq: 93701     NO_ACK
```

图 10 发送端

发送方计时器超时后发现没有收到期望的ACK，所以将93301对应的数据包重发，此时接收方收到了这个包才重发了ACK。

还有丢包的情况：

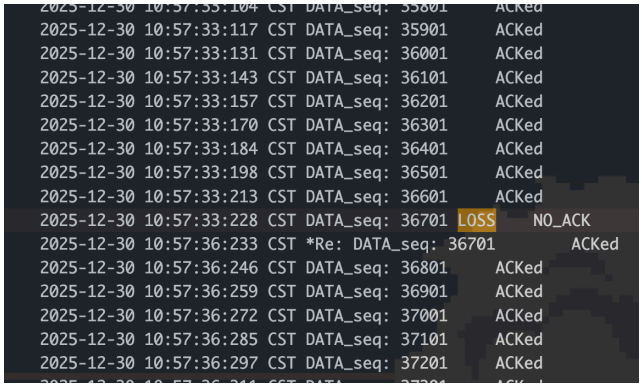


图 11 发送端

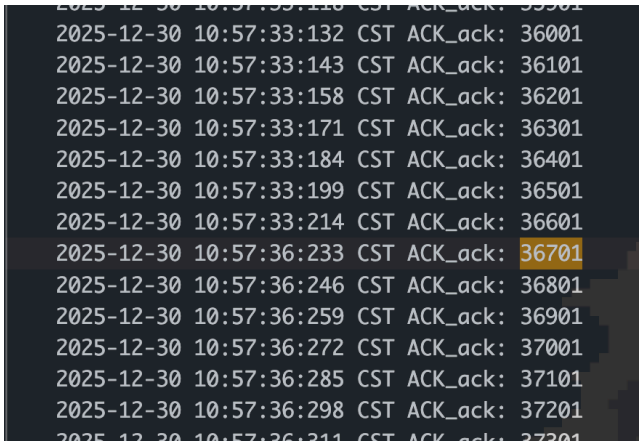


图 12 接收端

从 Log 文件可以观察到，在发生数据包或确认包丢失、延迟的情况下：

- 发送端在超时后能够自动重传数据包；
- 接收端对重传数据包正确返回确认，发送端随即停止重传；
- 上层接收到的数据内容连续、有序，未出现重复或缺失。

查看对应的recvData文件，还是为100000行，逻辑正确。

接下来开始实现对应的滑动窗口类协议，因为3.0虽然已经能够完成对错误的检测了，但是对信道的利用率太低了，在重传时信道是不工作的，滑动窗口解决了这个问题，流水线式地发送数据包，这里根据老师的指导先实现Select Response协议。

4. Select Response

在 RDT 3.0 的停等协议中，发送端在任何时刻只能发送一个未确认的数据包，信道利用率较低。Selective Repeat（SR）协议通过引入滑动窗口机制，使发送端能够在等待确认的同时连续发送多个数据包，从而提高传输效率。

SR 的核心特点在于：

- **发送端仅重传发生错误或超时的数据包**；
- **接收端对失序到达的数据包进行缓存**，并在条件满足时按序交付上层。

本阶段协议采用**SR 接收端 + 滑动窗口发送端**的设计方式，整体结构如下：

- **发送端（TCP_Sender + SenderWindow）**
  - 维护发送窗口，限制在途分组数量；
  - 每个分组具有唯一序号（以字节流序号表示）；
  - 收到 ACK 后，按序号确认对应分组并滑动窗口；
  - 支持窗口满时阻塞发送，防止发送端过载。
- **接收端（TCP_Receiver + ReceiverWindow）**
  - 维护接收窗口，允许乱序分组到达；
  - 对所有正确接收的分组立即返回 ACK；

- 仅当按序分组到达时才交付数据；
- 连续按序分组可一次性交付，窗口前移。

注：窗口中的数据结构为自己定义的SenderStru和ReceiveStru

发送端实现分析（TCP_Sender）

1. 分组序号设计

```
tcpH.setTh_seq(dataIndex * appData.length + 1);
```

发送端使用**字节流序号**来表示分组编号，每个分组的序号间隔为数据长度。这种方式兼容 TCP 报文头结构，同时在实验环境中可等价表示“分组序号”。

2. 滑动窗口控制与阻塞机制

发送端维护一个固定大小的发送窗口（16），当窗口已满时：

```
while (flag == WindowFlag.FULL.ordinal()) { }
```

发送端将阻塞等待，直到收到 ACK 并滑动窗口。这一机制保证了：

- 在途分组数量始终受控；
- 避免发送端无限制发送导致缓冲区溢出。

（虽然使用了忙等方式，但在功能层面满足实验要求。）

3. 分组入窗与发送

```
window.pushPacket(tcpPack.clone());
window.sendPacket(this, client, 3000, 3000);
```

- 每个分组在发送前都会**深拷贝**，避免对象复用导致内容被覆盖；
- 分组进入发送窗口后立即发送；
- 计时器的具体管理由 `SenderWindow` 统一负责，为后续 SR / 拥塞控制扩展提供了良好接口。

4. ACK 处理机制

```
int currentAck = ackQueue.poll();
window.ackPacket(currentAck);
```

发送端对接收端返回的 ACK 进行解析，并将其交由窗口模块处理。
ACK 的语义是“确认某一个具体分组的序号”，这是 SR 协议的核心特征之一。

接收端实现分析（TCP_Receiver）

1. 接收窗口与乱序缓存

```
int bufferResult = window.bufferPacket(recvPack);
```

接收端引入 `ReceiverWindow` 作为缓存结构，实现了以下功能：

- 对窗口范围内的分组进行缓存；
- 能区分：
  - 按序分组（IS_BASE）
  - 乱序但合法分组（ORDERED）
  - 重复分组（DUPLICATE）

这使得接收端不再要求分组严格按序到达，是 SR 协议相较 GBN 的关键改进。

2. 按分组确认（Per-packet ACK）

```
tcpH.setTh_ack(recvPack.getTcpH().getTh_seq());
reply(ackPacket);
```

无论分组是否乱序，只要校验和正确、分组合法，接收端都会**立即返回该分组对应的 ACK**：

- 防止发送端对已成功接收的分组进行不必要的重传；
- 使发送端能够精确定位丢失分组。

3. 有序交付与窗口前移

```
if (bufferResult == AckFlag.IS_BASE.ordinal()) {
    TCP_PACKET packet = window.getPacket();
    while (packet != null) {
        dataQueue.add(packet.getTcpS().getData());
        packet = window.getPacket();
    }
}
```

仅当当前接收到的是窗口 base 分组时，接收端才进行数据交付：

- 支持一次性交付多个连续按序分组；
- 确保上层应用始终接收到有序数据；
- 接收窗口随交付过程自动前移。

当前阶段实现特性总结

已实现功能：

✔ 支持滑动窗口发送
✔ 支持接收端乱序缓存
✔ 支持按分组确认（Selective ACK）
✔ 支持重复分组识别
✔ 支持连续按序交付
✔ 协议整体符合 Selective Repeat 的基本行为特征

在启用出错、丢包和延迟的信道条件下运行 SR 协议，从 Log 文件中可以观察到以下典型现象：

- 当某个数据包发生丢失或出错时，发送端仅在对应计时器超时后重传该数据包；
- 其他已经正确发送且被确认的数据包不会被重复重传；
- 接收端能够接收并缓存乱序到达的数据包，并在缺失的数据包补齐后一次性交付连续数据。

通过log文件可以看到很明显的滑动现象

```
2025-12-18 16:23:36:371 CST DATA_seq: 5501      ACKed
2025-12-18 16:23:36:384 CST DATA_seq: 5601      NO_ACK
2025-12-18 16:23:36:397 CST DATA_seq: 5701      NO_ACK
2025-12-18 16:23:36:408 CST DATA_seq: 5801      ACKed
2025-12-18 16:23:36:422 CST DATA_seq: 5901      ACKed
2025-12-18 16:23:36:435 CST DATA_seq: 6001      ACKed
2025-12-18 16:23:36:448 CST DATA_seq: 6101      ACKed
2025-12-18 16:23:36:459 CST DATA_seq: 6201      ACKed
2025-12-18 16:23:36:472 CST DATA_seq: 6301      ACKed
2025-12-18 16:23:36:485 CST DATA_seq: 6401      ACKed
2025-12-18 16:23:36:498 CST DATA_seq: 6501      ACKed
2025-12-18 16:23:36:511 CST DATA_seq: 6601      ACKed
2025-12-18 16:23:36:524 CST DATA_seq: 6701      ACKed
2025-12-18 16:23:36:537 CST DATA_seq: 6801      ACKed
2025-12-18 16:23:36:549 CST DATA_seq: 6901      ACKed
2025-12-18 16:23:36:562 CST DATA_seq: 7001      ACKed
2025-12-18 16:23:36:574 CST DATA_seq: 7101      ACKed
2025-12-18 16:23:39:390 CST *Re: DATA_seq: 5601     ACKed
2025-12-18 16:23:39:390 CST DATA_seq: 7201      ACKed
2025-12-18 16:23:39:402 CST *Re: DATA_seq: 5701     ACKed
```

图 13 发送端

```
2025-12-18 16:23:36:372 CST ACK_ack: 5501
2025-12-18 16:23:36:385 CST ACK_ack: 5601   LOSS
2025-12-18 16:23:36:397 CST ACK_ack: -849829853 WRONG
2025-12-18 16:23:36:408 CST ACK_ack: 5801
2025-12-18 16:23:36:423 CST ACK_ack: 5901
2025-12-18 16:23:36:436 CST ACK_ack: 6001
2025-12-18 16:23:36:449 CST ACK_ack: 6101
2025-12-18 16:23:36:460 CST ACK_ack: 6201
2025-12-18 16:23:36:473 CST ACK_ack: 6301
2025-12-18 16:23:36:486 CST ACK_ack: 6401
2025-12-18 16:23:36:498 CST ACK_ack: 6501
2025-12-18 16:23:36:512 CST ACK_ack: 6601
2025-12-18 16:23:36:524 CST ACK_ack: 6701
2025-12-18 16:23:36:537 CST ACK_ack: 6801
2025-12-18 16:23:36:550 CST ACK_ack: 6901
2025-12-18 16:23:36:563 CST ACK_ack: 7001
2025-12-18 16:23:36:575 CST ACK_ack: 7101
2025-12-18 16:23:39:390 CST *Re: ACK_ack: 5601
2025-12-18 16:23:39:391 CST ACK_ack: 7201
2025-12-18 16:23:39:402 CST ACK_ack: 5701
```

图 14 接收端

```
2025-12-18 16:24:20:940 CST DATA_seq: 92801     ACKed
2025-12-18 16:24:20:953 CST DATA_seq: 92901     ACKed
2025-12-18 16:24:20:965 CST DATA_seq: 93001 WRONG   NO_ACK
2025-12-18 16:24:20:977 CST DATA_seq: 93101     ACKed
2025-12-18 16:24:20:990 CST DATA_seq: 93201     ACKed
2025-12-18 16:24:21:003 CST DATA_seq: 93301     ACKed
2025-12-18 16:24:21:016 CST DATA_seq: 93401     ACKed
2025-12-18 16:24:21:027 CST DATA_seq: 93501     ACKed
2025-12-18 16:24:21:040 CST DATA_seq: 93601     ACKed
2025-12-18 16:24:21:052 CST DATA_seq: 93701     ACKed
2025-12-18 16:24:21:066 CST DATA_seq: 93801     ACKed
2025-12-18 16:24:21:079 CST DATA_seq: 93901     ACKed
2025-12-18 16:24:21:092 CST DATA_seq: 94001     ACKed
2025-12-18 16:24:21:105 CST DATA_seq: 94101     ACKed
2025-12-18 16:24:21:117 CST DATA_seq: 94201     ACKed
2025-12-18 16:24:21:129 CST DATA_seq: 94301     ACKed
2025-12-18 16:24:21:141 CST DATA_seq: 94401     ACKed
2025-12-18 16:24:21:153 CST DATA_seq: 94501     ACKed
2025-12-18 16:24:23:970 CST *Re: DATA_seq: 93001        ACKed
2025-12-18 16:24:23:970 CST DATA_seq: 94601     ACKed
```

图 15 发送端

```
2025-12-18 16:24:20:953 CST ACK_ack: 92901
2025-12-18 16:24:20:978 CST ACK_ack: 93101
2025-12-18 16:24:20:990 CST ACK_ack: 93201
2025-12-18 16:24:21:003 CST ACK_ack: 93301
2025-12-18 16:24:21:016 CST ACK_ack: 93401
2025-12-18 16:24:21:028 CST ACK_ack: 93501
2025-12-18 16:24:21:040 CST ACK_ack: 93601
2025-12-18 16:24:21:053 CST ACK_ack: 93701
2025-12-18 16:24:21:066 CST ACK_ack: 93801
2025-12-18 16:24:21:079 CST ACK_ack: 93901
2025-12-18 16:24:21:093 CST ACK_ack: 94001
2025-12-18 16:24:21:106 CST ACK_ack: 94101
2025-12-18 16:24:21:118 CST ACK_ack: 94201
2025-12-18 16:24:21:129 CST ACK_ack: 94301
2025-12-18 16:24:21:142 CST ACK_ack: 94401
2025-12-18 16:24:21:153 CST ACK_ack: 94501
2025-12-18 16:24:23:970 CST ACK_ack: 93001
2025-12-18 16:24:23:971 CST ACK_ack: 94601
```

图 16 接收端

通过对 `recvData` 文件的检查，可以确认最终交付的数据内容完整、有序，未出现重复或缺失。

5. TCP

在引入 TCP 拥塞控制机制之前，首先实现一个**不包含拥塞控制的 TCP 可靠传输协议**。
该阶段的目标是验证在仅依赖重传机制和滑动窗口的情况下，协议是否能够在不可靠信道上实现正确的数据传输，并为后续 Tahoe 与 Reno 拥塞控制算法的引入奠定基础。

本阶段实现的 TCP 具有如下特征：

- **发送端采用 GBN（Go-Back-N）风格的滑动窗口**
- **接收端采用 SR（Selective Repeat）缓存乱序数据**

- **ACK 使用累计确认，语义为“最后按序交付数据包的序列号”**
- **未引入任何拥塞控制算法（cwnd 恒定）**

系统总体设计

在无拥塞控制的 TCP 实现中，发送端与接收端的职责划分如下：

发送端（Sender）

发送端负责：

- 维护固定大小的发送窗口（窗口大小为 16）；
- 将应用层数据封装为 TCP 报文并按序发送；
- 接收 ACK 后滑动窗口；
- 通过统一计时器实现超时重传。

发送端并不根据网络状况动态调整发送速率，而是始终在窗口允许的范围内尽可能发送数据。

接收端（Receiver）

接收端负责：

- 校验接收报文的正确性；
- 对乱序到达的报文进行缓存；
- 按序交付数据；
- 向发送端返回累计确认 ACK。

接收端采用 Selective Repeat 的方式缓存窗口内乱序到达的数据分组，从而减少不必要的重传。

发送端滑动窗口机制（GBN）

发送窗口结构

发送端使用 `LinkedBlockingDeque` 维护发送窗口：

```
private final LinkedBlockingDeque<SenderStru> window;
```

窗口大小固定为 16：

```
private final int size = 16;
```

发送端在窗口未满时允许继续发送数据，否则阻塞等待 ACK 到达：

```
public boolean isFull() {
    return window.size() >= size;
}
```

数据发送流程

当应用层调用 `rdt_send()` 时，发送端执行如下步骤：

1. 根据应用层数据计算 TCP 序列号；
2. 封装 TCP 报文并计算校验和；
3. 将报文加入发送窗口；
4. 立即通过不可靠信道发送该报文。

对应核心代码如下：

```
window.putAndSend(tcpPack.clone());
```

在 `putAndSend()` 中，当窗口由空变为非空时，会启动统一的超时计时器。

超时重传机制

发送端采用 **单一计时器（base timer）**，即只对窗口最左侧（base）报文进行计时。

当发生超时时，执行如下操作：

- 重传窗口中所有尚未被确认的报文；
- 重新启动计时器。

```
public void sendWindow(){
    for(SenderStru pack : window){
        if (!pack.isAcked()) {
            sender.udt_send(pack.getPacket());
        }
    }
}
```

该策略符合 GBN 协议中“从 base 开始重传”的基本思想。

ACK 处理与窗口滑动

ACK 语义定义

本实验中 ACK 的语义定义为：

> **ACK 表示“最后一个被按序正确接收并交付的数据包的序列号”。**

该定义与 TCP 的累计确认机制一致。

发送端 ACK 处理逻辑

发送端在收到 ACK 后，会执行累计确认：

```java
while (true) {
    SenderStru first = window.peekFirst();
    if (first == null) break;

    int seq = first.getPacket().getTcpH().getTh_seq();
    if (seq <= ack) {
        window.pollFirst();
        first.ackPacket();
        remove = true;
    } else {
        break;
    }
}
```

只要窗口左侧报文的序列号小于等于 ACK，即可将其从窗口中移除。

当窗口成功前移后：

- 重置计时器；
- 唤醒可能因窗口已满而阻塞的发送线程。

接收端代码沿用之前的Select Response修改成的，有乱序包缓存和延迟 500ms

累积确认的能力，因此不需要修改。

保持 `tcpH.setTh_eflag((byte) 7)`，允许信道出错、丢包、延迟，运行 TCP 代码。

累计的ACK会在500ms后发出，正常只确认窗口最右侧的包

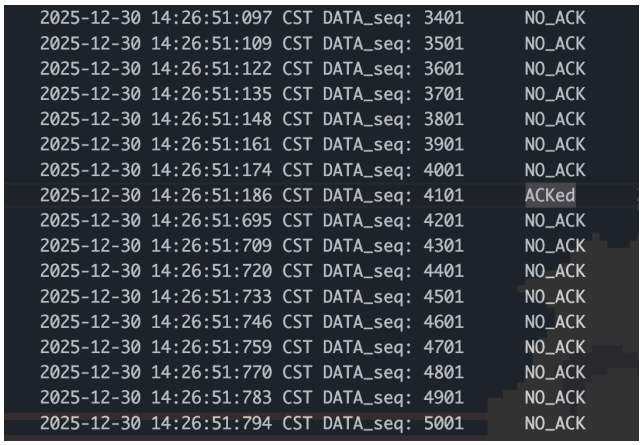


图 17 发送端

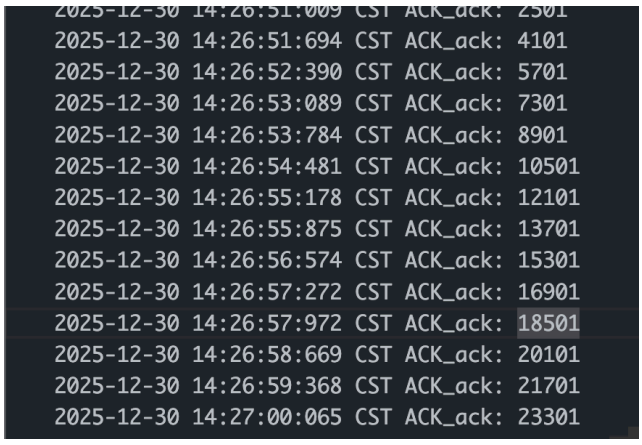
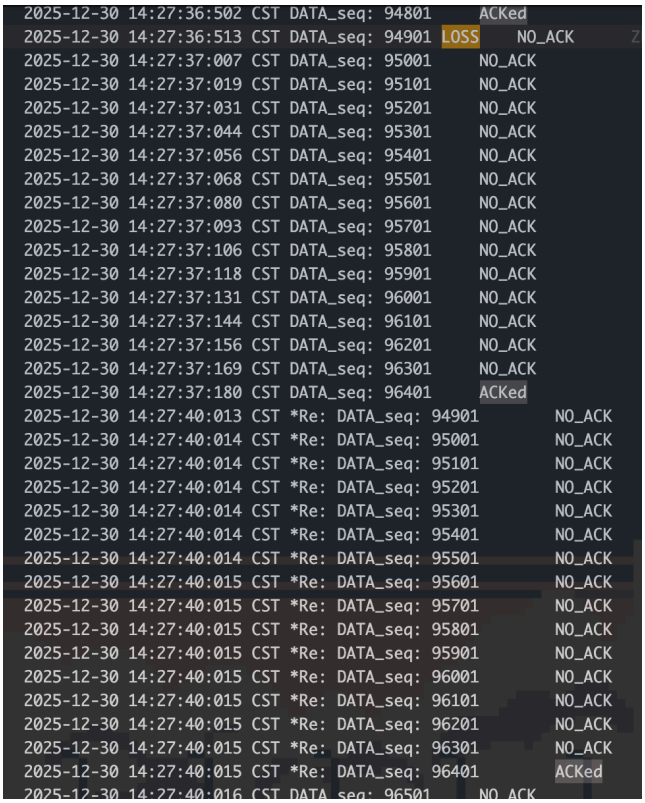


图 18 接收端

如果出现丢包后产生的重传，延迟3s之后对整个窗口进行重传。



```
2025-12-30 14:27:36:502 CST DATA_seq: 94801     ACKed
2025-12-30 14:27:36:513 CST DATA_seq: 94901 LOSS    NO_ACK
2025-12-30 14:27:37:007 CST DATA_seq: 95001     NO_ACK
2025-12-30 14:27:37:019 CST DATA_seq: 95101     NO_ACK
2025-12-30 14:27:37:031 CST DATA_seq: 95201     NO_ACK
2025-12-30 14:27:37:044 CST DATA_seq: 95301     NO_ACK
2025-12-30 14:27:37:056 CST DATA_seq: 95401     NO_ACK
2025-12-30 14:27:37:068 CST DATA_seq: 95501     NO_ACK
2025-12-30 14:27:37:080 CST DATA_seq: 95601     NO_ACK
2025-12-30 14:27:37:093 CST DATA_seq: 95701     NO_ACK
2025-12-30 14:27:37:106 CST DATA_seq: 95801     NO_ACK
2025-12-30 14:27:37:118 CST DATA_seq: 95901     NO_ACK
2025-12-30 14:27:37:131 CST DATA_seq: 96001     NO_ACK
2025-12-30 14:27:37:144 CST DATA_seq: 96101     NO_ACK
2025-12-30 14:27:37:156 CST DATA_seq: 96201     NO_ACK
2025-12-30 14:27:37:169 CST DATA_seq: 96301     NO_ACK
2025-12-30 14:27:37:180 CST DATA_seq: 96401     ACKed
2025-12-30 14:27:40:013 CST *Re: DATA_seq: 94901        NO_ACK
2025-12-30 14:27:40:014 CST *Re: DATA_seq: 95001        NO_ACK
2025-12-30 14:27:40:014 CST *Re: DATA_seq: 95101        NO_ACK
2025-12-30 14:27:40:014 CST *Re: DATA_seq: 95201        NO_ACK
2025-12-30 14:27:40:014 CST *Re: DATA_seq: 95301        NO_ACK
2025-12-30 14:27:40:014 CST *Re: DATA_seq: 95401        NO_ACK
2025-12-30 14:27:40:014 CST *Re: DATA_seq: 95501        NO_ACK
2025-12-30 14:27:40:015 CST *Re: DATA_seq: 95601        NO_ACK
2025-12-30 14:27:40:015 CST *Re: DATA_seq: 95701        NO_ACK
2025-12-30 14:27:40:015 CST *Re: DATA_seq: 95801        NO_ACK
2025-12-30 14:27:40:015 CST *Re: DATA_seq: 95901        NO_ACK
2025-12-30 14:27:40:015 CST *Re: DATA_seq: 96001        NO_ACK
2025-12-30 14:27:40:015 CST *Re: DATA_seq: 96101        NO_ACK
2025-12-30 14:27:40:015 CST *Re: DATA_seq: 96201        NO_ACK
2025-12-30 14:27:40:015 CST *Re: DATA_seq: 96301        NO_ACK
2025-12-30 14:27:40:015 CST *Re: DATA_seq: 96401        ACKed
2025-12-30 14:27:40:016 CST DATA_seq: 96501     NO_ACK
```

图 19 发送端

无拥塞控制的 TCP 在 SR 的基础上，通过以下机制进一步提升了系统的整体行为一致性与工程合理性：

- 发送端采用单计时器的滑动窗口重传策略，降低了计时器管理复杂度；
- 接收端结合失序缓存与累积确认，提高了传输效率并减少 ACK 数量；
- 在复杂不可靠信道条件下，仍能保证可靠、有序的数据交付。

6. TCP Tahoe

在发送端窗口模块 `SenderWindow` 中引入 TCP Tahoe 的核心拥塞控制机制，包括：**慢启动（Slow Start）、拥塞避免（Congestion Avoidance）、快重传（Fast Retransmit）。Tahoe 的特点是：一旦判断拥塞（超时或 3dupACK），发送端都会将拥塞窗口** cwnd 退回到 1，重新进入慢启动阶段（不包含 Reno 的快恢复）。

关键变量设计

```
private int cwnd = 1;
private double cwnd_d = 1.0;
private int ssthresh = 16;
```

有效发送窗口为：

```
private int effectiveCwnd() {
    return Math.min(cwnd, size);
}
```

慢启动与拥塞避免

当收到新的累计确认时，根据当前阶段调整 `cwnd`：

```
if (cwnd < ssthresh) {
    cwnd++;
    cwnd_d = (double)cwnd;
} else {
    cwnd_d += 1.0 / cwnd;
    cwnd = (int)cwnd_d;
}
```

该实现满足：

- 慢启动阶段：`cwnd` 近似指数增长
- 拥塞避免阶段：`cwnd` 线性增长

丢包检测与重传机制

超时重传（Timeout Retransmit）

当定时器到期，发送端认为发生严重拥塞：

```
private void onTimeout(){
    ssthresh = Math.max(1, cwnd / 2);
    cwnd = 1;
    cwnd_d = 1.0;
    lastAck = -1;
    dupAckCount = 0;
}
```

随后只重传窗口最左侧的 base 包：

```
private void resendBase(){
    SenderStru first = window.peekFirst();
    if (first != null) {
        sender.udt_send(first.getPacket());
    }
}
```

快重传（Fast Retransmit）

通过统计重复 ACK 判断丢包：

```
if (ack == lastAck) {
    dupAckCount++;
} else {
    lastAck = ack;
    dupAckCount = 1;
}
```

当收到 3 个重复 ACK 时，立即重传缺失数据包：

```
if (dupAckCount >= DUP_ACK_THRESHOLD) {
    ssthresh = Math.max(2, cwnd / 2);
    cwnd = 1;
    cwnd_d = 1.0;
    resendPacket(ack);
    resetTimer();
    dupAckCount = 0;
}
```

在日志中的体现为：

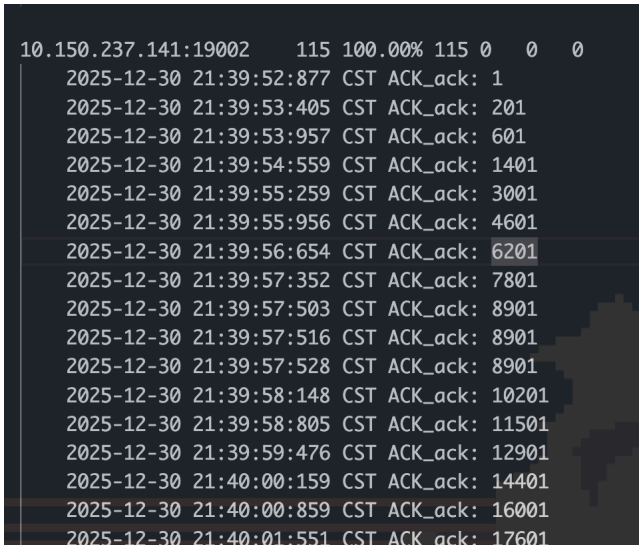


图 20 慢开始

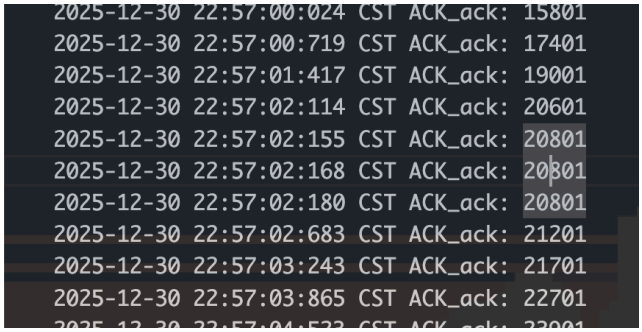


图 21 快重传

基本符合Tahoe的要求。

7. TCP Reno

Reno 则是在 Tahoe 的基础上加入了快恢复机制，即当连续收到 3 个相同的 ACK 时，ssthresh 减

半，cwnd 重置为 ssthresh + 3而不是 1。

修正

```
if (dupAckCount >= DUP_ACK_THRESHOLD) {
    ssthresh = Math.max(2, cwnd / 2);
    cwnd = ssthresh + 3;
    cwnd_d = 1.0;
    resendPacket(ack);
    resetTimer();
    dupAckCount = 0;
}
```

这样可以使得cwnd 在快恢复阶段线性增长，而不是慢开始阶段的指数增长。在平均情况下 Reno 的传输效率要高于 Tahoe。

二、实验总结

1. 未完成的项目

关键困难基本完成，实现了TCP的基本功能包括拥塞控制。

2. 迭代开发

实验中使用git进行版本控制，从RDT1.0开始进行迭代开发，使用不同的分支来代表不同的版本。在 TCP 协议实现过程中（从 RDT1.0 到 Reno），协议逻辑复杂、状态多、改动频繁，**Git 提供了一种“可回溯、可对比、可验证”的工程化手段**，使得协议演进过程可控、可解释、可复现。也可以帮助我们更好地理解TCP打发展以及不同协议TCP的差异。

```
commit af62219623cada3b31f85a910d4ab7a894f70bfa (HEAD -> TCP-Reno)
Author: Zhang Gavin <zhanggavin@ZhangdeMacBook-Air.local>
Date:   Tue Dec 30 22:55:51 2025 +0800

    窗口修改

commit 3ded45892924ea0175cc8a9579bb720a9992cc94
Author: Zhang Gavin <zhanggavin@ZhangdeMacBook-Air.local>
Date:   Mon Dec 29 14:57:05 2025 +0800

    增加了可视化内容

commit 1f92bfd0d131ca8ea059903dd3f9aee2a080d3f8 (origin/TCP-Reno)
Author: Zhang Gavin <zhanggavin@ZhangdeMacBook-Air.local>
Date:   Mon Dec 29 11:02:02 2025 +0800

    Reno具体实现以及测试结果

commit 7536c69f28b3d7433256bb6ec276f8dbd6192547 (origin/TCP-Tahoe)
Author: Zhang Gavin <zhanggavin@ZhangdeMacBook-Air.local>
Date:   Mon Dec 29 10:56:20 2025 +0800

    测试结果

commit 3c2ddb6981ed67384ed6de87edcb5a501ebb2532
Author: Zhang Gavin <zhanggavin@ZhangdeMacBook-Air.local>
Date:   Mon Dec 29 10:52:31 2025 +0800

    Tahoe实现，加入拥塞控制

commit bc91c90457b33a37d5e01eb0f43512ff5a6f835a (origin/TCP)
Author: Zhang Gavin <zhanggavin@ZhangdeMacBook-Air.local>
Date:   Sun Dec 28 15:02:49 2025 +0800

    接收端沿用SR逻辑，发送端改为GBN逻辑
```

图 22 git log

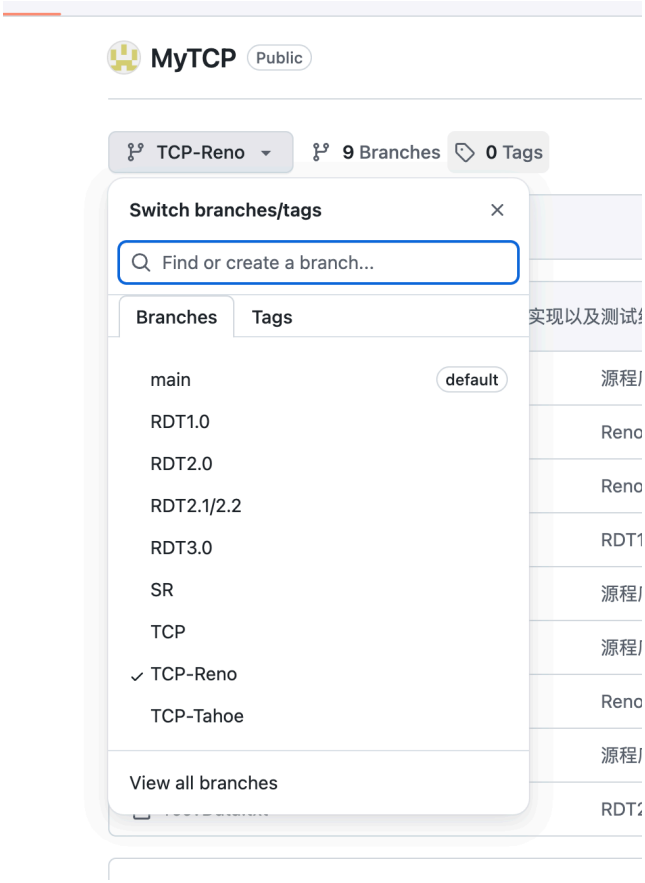


图 23 GitHub仓库

3. 已解决的主要问题

实验中经常会写着写着就写迷糊了，自己也不知道改如何进行下去了：(

实验中遇到最大的困难即最后在实现到Tahoe时超时重传

在 TCP Tahoe 与 TCP Reno 阶段的实现过程中，**超时重传与发送窗口状态管理之间的关系**是实验中的一个关键难点。如果处理不当，容易出现超时后重传范围过大、重复触发重传或错误触发快重传等问题。

在本实验中，发送端采用了滑动窗口与发送缓存相结合的设计方式：发送窗口用于描述当前允许发送的数据范围，而所有已发送但尚未确认的数据包均被保存在发送缓存中。针对超时事件的处理，实验中采取了如下解决策略。

当发送端检测到超时事件后，将其视为严重拥塞信号，首先按照 TCP Tahoe 的规则将慢开始阈值设置为当前拥塞窗口的一半，并将拥塞窗口大小重置为 1。在窗口管理层面，发送端通过**清空当前发送窗口的方式，使窗口状态回退到仅包含窗口左沿对应的数据包**。需要强调的是，窗口的清空并不意味着数据丢失，其余尚未确认的数据包仍然完整地保存在发送缓存中，仅暂时不参与发送。

随后，发送端仅从发送缓存中选取窗口左沿（base）对应的数据包进行重传，并重新启动超时计时器以跟踪新的窗口左沿。后续数据包是否重新进入发送窗口并被发送，完全由慢开始过程以及接收端返回的确认报文驱动，而非在超时后一次性重传窗口内的所有数据包。

通过上述方式，实验实现避免了超时后无差别重传整个窗口所带来的问题，例如接收端瞬间返回大量确认、错误触发重复 ACK 计数以及在 TCP Reno 阶段误触发快重传等情况。同时，该实现方式在行为语义上严格符合 TCP Tahoe 的设计要求：**超时后仅重传窗口左沿的数据包，并通过慢开始机制逐步恢复发送速率**。

实验结果表明，在该策略下，发送端能够在发生超时的情况下稳定收敛发送窗口大小，接收端交付的数据始终保持有序且无重复，相关行为与 TCP Tahoe 与 TCP Reno 的协议预期一致。

4. 系统性建议

没有把这个当作一个工程来做，没有把需求的优先级进行排序来设计对应的程序语言。建议在实验中进一步明确：哪些行为属于协议层面的强约束，哪些属于允许多种实现方式的工程细节。通过对这两类内容进行区分，在保证协议行为正确的前提下，采用更加合理和清晰的工程实现方式。

实验的整体需求不是很明确，希望能有更准确的需求以及规格说明。